Iteration B1: Validating!

While playing with the results of iteration A1, our client noticed something. If she entered an invalid price or forgot to set up a product description, the application happily accepted the form and added a line to the database. A missing description is embarrassing, and a price of $0.00 costs her actual money, so she asked that we add validation to the application. No product should be allowed in the database if it has an empty title or description field, an invalid URL for the image, or an invalid price.

So where do we put the validation? The model layer is the gatekeeper between the world of code and the database. Nothing to do with our application comes out of the database or gets stored into the database that doesn't first go through the model. This makes models an ideal place to put validations; it doesn't matter whether the data comes from a form or from some programmatic manipulation in our application. If a model checks it before writing to the database, the database will be protected from bad data.

Let's look at the source code of the model class (in `app/models/product.rb`):

```
class Product < ApplicationRecord
end
```

Adding our validation should be fairly clean. Let's start by validating that the text fields all contain something before a row is written to the database. We do this by adding some code to the existing model:

```
validates :title, :description, :image_url, presence: true
```

The `validates` method is the standard Rails validator. It checks one or more model fields against one or more conditions.

`presence: true` tells the validator to check that each of the named fields is present and that its contents aren't empty. The following screenshot shows what happens if we try to submit a new product with none of the fields filled in. Try it by visiting `http://localhost:3000/products/new` and submitting the form without